Arquitetura do SaaS

WhatsApp + AWS Lambda + Groq AI

Documento Técnico — Visão Geral da Infraestrutura

---

1. Por que Oracle Cloud Free Tier?

Ao hospedar o WAHA (gateway WhatsApp), você precisa de um servidor rodando 24/7 sem custo e sem risco de cobrança surpresa. O Oracle Cloud Free Tier é a melhor opção disponível hoje por três razões principais:

- Always Free — recursos que nunca expiram, sem contador regressivo de 12 meses
- VM ARM Ampere A1 com até 4 OCPUs e 24 GB de RAM gratuitamente
- Sem sleep por inatividade — o bot responde imediatamente a qualquer hora

Comparativo entre plataformas

Critério	Oracle Cloud	Railway	Render/Heroku	AWS EC2
Gratuito para sempre	✅ Always Free	❌ Não (trial/crédito)	⚠️ Parcial (sleep)	❌ 12 meses
RAM disponível	Até 24 GB (ARM)	Variável (crédito)	512 MB – 1 GB	1 GB
Sleep por inatividade	❌ Nunca	❌ Não	✅ Sim (15 min)	❌ Não
Risco de cobrança	🟢 Baixo	🟠 Médio	🟠 Médio	🔴 Alto
Controle total da VM	✅ Sim	❌ Não	❌ Não	✅ Sim
Ideal para bots 24/7	✅ Sim	⚠️ Com risco	❌ Não	⚠️ Com risco

Plataformas como Render suspendem o serviço após 15 minutos de inatividade — inaceitável para um bot de WhatsApp. Railway encerrou seu plano gratuito. AWS EC2 expira em 12 meses

e tem alto risco de cobranças inesperadas. Oracle é a única opção verdadeiramente gratuita e permanente com recursos generosos.

---

2. Fluxo Completo da Arquitetura

Arquitetura Atual (Telegram)

```
User → Telegram Bot → Webhook → API Gateway (AWS) → Lambda → DynamoDB
```

Nova Arquitetura (WhatsApp + IA)

```
User (WhatsApp)
     │
     ▼
 WAHA (Oracle Cloud VM)          ← substitui o Telegram Bot
     │  recebe mensagem e dispara webhook POST
     ▼
 API Gateway (AWS)               ← mantido igual
     │
     ▼
 Lambda (AWS)                    ← ajuste no parsing + chamada Groq
     │  1. extrai payload do WAHA
     │  2. chama Groq API com a mensagem
     │  3. recebe JSON estruturado
     │  4. salva no DynamoDB
     ▼
 DynamoDB (AWS)                  ← mantido igual
```

O API Gateway, Lambda e DynamoDB permanecem 100% intactos. A única troca é o Telegram Bot pelo WAHA, e a adição da chamada à Groq API dentro do Lambda existente.

---

3. O que Muda no Lambda

3.1 Diferença no Payload

O JSON que chega no Lambda muda de formato. Veja a comparação:

Payload do Telegram (antes):

```
{
  "message": {
```

```
    "from": { "id": 123456, "username": "joao" },
    "text": "mano acabei de pagar 200 num moletom no debito"
  }
}
```

Payload do WAHA (depois):

```
{
  "event": "message",
  "session": "default",
  "payload": {
    "id": "AAAABBBB112233",
    "from": "5511999999999@c.us",
    "body": "mano acabei de pagar 200 num moletom no debito",
    "timestamp": 1710000000
  }
}
```

A mudança no Lambda é simples: trocar event.message.text por event.payload.body (ou body.payload.body dependendo de como o API GW passa o evento).

---

4. Processamento de Linguagem Natural com Groq

Por que no Lambda?

Sim, o Lambda é o lugar certo para isso. Ele já recebe a mensagem, tem acesso à internet e é o ponto central de orquestração. A chamada à Groq acontece dentro do mesmo fluxo, antes de salvar no DynamoDB.

4.1 Exemplo de Chamada à Groq API

```
// Dentro do Lambda Handler
const userMessage = payload.body;

const groqResponse = await
fetch('https://api.groq.com/openai/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${process.env.GROQ_API_KEY}`,
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    model: 'llama3-8b-8192',
    messages: [
      {
        role: 'system',
        content: `Você é um extrator de dados financeiros.
```

```
Analise a mensagem e retorne SOMENTE um JSON no formato:
{ "intent": "expense"|"income", "amount": number,
  "category": string, "method": "pix"|"debito"|"credito"|"dinheiro" }
Sem explicações, apenas o JSON.`
      },
      { role: 'user', content: userMessage }
    ],
    temperature: 0.1
  })
});
```

4.2 Exemplo de Entrada e Saída

Mensagem do usuário	JSON gerado pelo Groq
"mano acabei de pagar 200 num moletom no debito"	{ "intent": "expense", "amount": 200.00, "category": "vestuario", "method": "debito" }
"recebi 1500 de freela hoje no pix"	{ "intent": "income", "amount": 1500.00, "category": "freela", "method": "pix" }

4.3 Salvando no DynamoDB

```
const parsed = JSON.parse(groqResponse.choices[0].message.content);

await dynamodb.put({
  TableName: 'financas',
  Item: {
    userId:    payload.from,          // '5511999999999@c.us'
    timestamp: Date.now(),
    intent:    parsed.intent,         // 'expense'
    amount:    parsed.amount,         // 200.00
    category:  parsed.category,       // 'vestuario'
    method:    parsed.method,         // 'debito'
    rawMessage: userMessage           // mensagem original
  }
}).promise();
```

---

5. Resumo Executivo

Componente	Papel na arquitetura
Oracle Cloud VM	Hospeda o WAHA (gateway WhatsApp) 24/7 gratuitamente
WAHA	Recebe mensagens do WhatsApp e dispara webhooks POST
API Gateway (AWS)	Recebe o webhook e roteia para o Lambda — sem mudanças
Lambda (AWS)	Extrai payload WAHA + chama Groq API + salva no DynamoDB
Groq API (LLM)	Transforma linguagem natural em JSON estruturado
DynamoDB (AWS)	Armazena os dados financeiros estruturados — sem mudanças

O que você mantém intacto: API Gateway, DynamoDB, estrutura do Lambda.
O que você troca: Telegram Bot → WAHA (Oracle Cloud).
O que você adiciona: chamada à Groq API dentro do Lambda para NLP.